MECHANIQ SETUP & RUN GUIDE

Complete Step-by-Step Instructions for Running the App from GitHub

Platform: React Native (Expo SDK 57) + Node.js + Python

Updated: October 2026

Target OS: Android (arm64-v8a)

Welcome to **MechanIQ**! This guide provides the complete setup procedure for team members who pull the repository from GitHub. MechanIQ is a dual-dashboard automotive application featuring separate experiences for **Customers** and **Mechanics**, connected to three local backend microservices for sound diagnostics, OBD data, and image recognition.

1 Architecture & Microservices Overview

Before launching the app, understand the four primary components that make up the MechanIQ ecosystem:

Component	Technology	Port / URL	Description
Mobile App	React Native 0.86, Expo 57, Expo Router	`Metro: 8081`	Dual-dashboard client (Customer & Mechanic flows).
Picture Search API	Node.js / Express / Google Gemini Vision	`Port 3002`	Identifies car components from uploaded or captured images.
OBD-II Telemetry API	Node.js / Express	`Port 5002`	Simulates and reads diagnostic PIDs and fault codes.
Sound Detection API	Python / FastAPI / TensorFlow / Librosa	`Port 8000`	AI model classifying engine audio anomalies from recordings.
Auth & Database	Firebase Auth & Realtime Database	`Cloud`	Role-based profile storage, live messaging, and appointments.

2 Prerequisites & Required Software

Make sure your workstation has the following installed before starting:

- **Node.js:** Version `20.x` or `22.x` (LTS recommended).
- **Python:** Version `3.10` or `3.11` (required for the Sound Detection service).
- **Java JDK:** JDK 17 (e.g. Eclipse Adoptium / Temurin 17). *Note: Java 21+ or 25 may trigger restricted library warnings during Android CMake builds.*
- **Android Studio & SDK:**
  - Android SDK Platform 36 and Build-Tools 36.0.0.
  - NDK version `27.1.12297006`.

    - `platform-tools` added to your system `PATH` (so `adb` is recognized).
- **Physical Android Phone:** Connected via USB with **Developer Options** and **USB Debugging** turned ON.

3 Step 1: Clone Repository & Install Frontend Packages

Clone the repository and install npm packages in the project root directory:

```
# 1. Clone repository
git clone <YOUR_GITHUB_REPO_URL>
cd m

# 2. Install dependencies
npm install
```

> **Module Shims Included**
> Platform-specific shims for `@react-native-community/netinfo`, `expo-crypto`, `expo-notifications`, and `expo-secure-store` are bundled in `src/shims/` and configured in `metro.config.js`. No manual native linking or additional installs are required.

4 Step 2: Configure Environment Variables

Create two environment configuration files in the project root folder (`m/`):

File A: `.env` (Default API Endpoints)

```
# Default localhost endpoints mapped via USB reverse tunnels
EXPO_PUBLIC_PICTURE_SEARCH_API_URL=http://127.0.0.1:3002
EXPO_PUBLIC_OBD_API_URL=http://127.0.0.1:5002
EXPO_PUBLIC_SOUND_DIAGNOSIS_API_URL=http://127.0.0.1:8000
```

File B: `.env.local` (Secret Keys & Local Runtime Paths)

```
# AI Services & Backend Configuration
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
GEMINI_MODEL=gemini-3.8-flash
SERPAPI_API_KEY=<YOUR_SERPAPI_KEY>

# Python executable path for Sound Detection (system 'python' or venv path)
MECHANIQ_SOUND_PYTHON=python

# Device endpoints
EXPO_PUBLIC_PICTURE_SEARCH_API_URL=http://127.0.0.1:3002
EXPO_PUBLIC_OBD_API_URL=http://127.0.0.1:5002
EXPO_PUBLIC_SOUND_DIAGNOSIS_API_URL=http://127.0.0.1:8000
```

5 Step 3: Setup the Sound Detection Python Backend

The AI sound diagnosis model runs on FastAPI and TensorFlow. Set up its virtual environment:

```
# Navigate to sound detection backend
cd backend/sound-detection

# Create and activate Python virtual environment
python -m venv venv
.\venv\Scripts\activate       # On Windows PowerShell
# source venv/bin/activate    # On macOS / Linux

# Install required Python libraries
pip install -r requirements.txt

# Return to root directory
cd ../..
```

Audio Codec Note (m4a support)

Android and iOS record sound files in `.m4a` format. Ensure `ffmpeg` is installed on your computer and available in your system `PATH` so audio decoding functions seamlessly.

6 Step 4: Start All Three Backend Services (One Command)

MechanIQ provides a built-in startup script that launches all microservices in parallel and confirms readiness:

```
# Run from project root (m/)
node scripts/start-backends.js
```

You will see console output confirming health status for all three services:

```
[READY] Picture Search is responsive at http://127.0.0.1:3002/health
[READY] OBD API is responsive at http://127.0.0.1:5002/health/ready
[READY] Sound Detection is responsive at http://127.0.0.1:8000/health/ready
ALL THREE BACKEND SERVICES ARE OPERATIONAL AND LISTENING ON 0.0.0.0
```

Keep This Terminal Open!

Leave this command running in its own terminal window while testing or developing. Closing it will stop the backend APIs and cause the app to display `fetch failed`.

7 Step 5: Setup USB Reverse Tunnels (Physical Phone)

To allow your physical phone to communicate with the local backends over USB without modifying IP addresses every time your Wi-Fi changes, run:

```
# Check that your device is detected
adb devices

# Forward backend ports and Metro bundler
adb reverse tcp:3002 tcp:3002
adb reverse tcp:5002 tcp:5002
adb reverse tcp:8000 tcp:8000
adb reverse tcp:8081 tcp:8081
```

Why Reverse Tunnels?

When the app on your phone calls `http://127.0.0.1:3002`, ADB immediately routes that packet across the USB cable to port 3002 on your computer. You never need to look up your laptop's local IP address or worry about Wi-Fi network isolation.

8 Step 6: Run the App on Your Phone

You can run the app in either Development Mode (Expo Metro) or Build a Standalone Release APK:

Option A: Development Mode (Metro Bundler)

```
# Start the Expo bundler
npx expo start -c

# In the Metro terminal, press 'a' to launch on your connected Android device
```

Option B: Building & Installing the Release APK

```
# 1. Point JAVA_HOME to JDK 17
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"   # Or your Adoptium JDK 17
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"

# 2. Compile Release APK with Gradle
cd android
.\gradlew.bat :app:assembleRelease --no-daemon

# 3. Install directly onto your connected device
adb install -r app/build/outputs/apk/release/app-release.apk
```

9 Step 7: Testing Customer & Mechanic Roles

MechanIQ enforces strict role-based access control. Here is how each user type functions:

Feature	Customer (General User)	Mechanic (Specialist)
Sign Up Path	Welcome Screen → *"I'm a Customer"*	Welcome Screen → *"I'm a Mechanic"*
Login Identifier	Personal Email (e.g. `john@gmail.com`)	**App-Generated Username** (e.g. `mike@mechanic`)
Communication Email	Same as Login Email	Collected separately during sign-up for notifications.
Destination Screen	**Customer Dashboard** (`/(customer)`)	**Mechanic Dashboard** (`/(mechanic)`)
Available Features	Diagnostics, Fuel Top-up, Marketplace, Store, Profile.	Incoming Jobs, Live Diagnostics, Customer Roster, Workshop Profile.
Access Guard	Blocked from accessing Mechanic tabs.	Blocked from accessing Customer tabs.

Mechanic Username Flow Details:

1. On sign-up Step 1, enter the mechanic's name (e.g. *Mike*).
2. The system dynamically generates the username: `mike@mechanic`. (If taken, numbers are appended automatically e.g. `mike1@mechanic`).
3. Provide communication email, workshop details, experience, and certifications.
4. In Step 3, choose a strong password and submit.
5. A success dialog displays the assigned username: `mike@mechanic`.
6. Tap **Sign In Now** → enter `mike@mechanic` and password → routes directly to the Mechanic Dashboard.

10 Troubleshooting & Common Issues

Issue: "Fetch failed" or "Cannot reach sound detection backend"

Cause:

Either the backend services are not running, or USB reverse tunnels were disconnected when unplugging the cable.

Fix:

Run `node scripts/start-backends.js` in terminal 1, and run `adb reverse tcp:3002 tcp:3002; adb reverse tcp:5002 tcp:5002; adb reverse tcp:8000 tcp:8000` in terminal 2.

Issue: "A restricted method in java.lang.System has been called" during APK build

Cause:

Gradle is using Java 24 or 25, which enforces strict restriction warnings that Android Prefab treats as errors.

Fix:

Switch your `JAVA_HOME` to JDK 17 before building: `$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"`.

Issue: "Execution policies prevent running npx or scripts" on Windows PowerShell

Cause:

Windows restricts unsigned `.ps1` execution by default.

Fix:

Use `npx.cmd` instead of `npx`, or run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.